censtrunc

Censored and Truncated Normal Regression with Arbitrary Thresholds

Author: <Your Name>
Package: `censtrunc` v0.1.0

Abstract

`censtrunc` is a Python package for maximum-likelihood estimation of censored and truncated normal regression models with **arbitrary left and right thresholds**, plus the Heckman (1979) sample-selection model. It generalises the classical Type-1 Tobit (Tobin, 1958) and the truncated normal regression to two-sided limits, exposes a scikit-learn-style API, a statsmodels-style results summary, marginal effects, likelihood-ratio tests, and patsy-style formulas. Estimation uses Olsen's (1978) globally concave reparameterisation for the censored case. The report opens with a survey of what is and is not available in Python for these models today, then states the statistical model and presents six worked examples.

1. Python landscape, and what `censtrunc` adds

Compared to R -- where `survival::survreg` (wrapped by `AER::tobit`) covers the censored case, `truncreg::truncreg` covers truncation, and `sampleSelection::selection` covers Heckman selection with both estimators -- the Python ecosystem for these models is fragmented.

Package	Censored / Tobit	Truncated	Heckit	
statsmodels	no public Tobit class	not in the public API	not implemented	The pla firs doc gap
scikit-learn	—	—	—	No dep var
linearmodels	—	—	—	Par spe

Package	Censored / Tobit	Truncated	Heckit	
lifelines	left-censored Weibull / log-normal AFT	—	—	Sur haz coe awl nor
scikit-survival	right-censored, AFT-style	—	—	Sar fra
py4etrics	`Tobit` with arbitrary L, R	`Truncreg` with arbitrary L, R	two-step `Heckit` (`method='mle'` silently ignored)	Edu pac ma no for
stnwanekezie/TobitRegression	a single `Tobit` class on top of `statsmodels.OLS` with Olsen's reparameterisation	—	—	425 su onl
PyMC / bambi	Bayesian via `Censored`	Bayesian via `Truncated`	possible to hand-code	Bay MC
marginaleffects	—	—	—	Slo too fitt

What `censtrunc` adds on top

- **Two-sided censoring and truncation** with arbitrary L, R in every estimator (one-sided becomes a special case via `left=None` or `right=None`).
- **Heckman selection with joint MLE** (matching `sampleSelection::selection`), not just the two-step. The two-step ships with a paired bootstrap for the proper Heckman-corrected SEs.
- **Marginal effects** through a single `get_margeff` method mirroring statsmodels' API, for all three estimators. Censored models get the three region-probability kinds (`prob-left`, `prob-interior`, `prob-right`) in addition to `latent` / `censored` / `truncated`; Heckit gets the four Heckman quantities (`latent`, `conditional`, `unconditional`, `prob-selected`), with variables matched by name across the outcome and selection equations.
- **Likelihood-ratio test** in two forms: between any pair of fitted models, or `model.lr_test(hypotheses)` with statsmodels-style hypothesis strings (`'(x1 = 0), (x2 = x3)'`, `'2*x1 + x4 = 1'`).
- **Patsy formulas** on all three classes; the same model can be fit either by passing arrays `(X, y[, Z])` or via `Model.from_formula(formula, data=df, ...)`.

- A unified `predict()` returning any combination of **six** quantities (three conditional means + three region probabilities) via a one-letter `kind` string -- e.g. `'hctlmr'` for all of them, `'lmr'` for just the probabilities.
- **Cross-validation against external implementations** in the test suite: OLS-equivalence and probit-equivalence limits, R's `survival::survreg`, `truncreg::truncreg`, `sampleSelection::selection`, and `py4etrics.Heckit`.

2. Statistical Model

2.1 Latent regression

All models share the latent linear specification with normal errors:

$$Y^* = X'\beta + e, \qquad e \mid X \sim \mathcal{N}(0, \sigma^2).$$

What differs is the rule linking the latent Y^* to the observed data.

2.2 Censored model

Values outside $[L, R]$ are **clipped** to the nearest threshold:

$$Y = \min\big(R,\ \max(L,\ Y^*)\big).$$

The log-likelihood combines the probability mass at each threshold with the normal density on the interior. With $\alpha_L = (L - X'\beta)/\sigma$ and $\alpha_R = (R - X'\beta)/\sigma$,

$$\ell(\beta, \sigma) = \sum_{Y_i = L} \log \Phi(\alpha_{L,i}) + \sum_{Y_i = R} \log\big[1 - \Phi(\alpha_{R,i})\big] + \sum_{L < Y_i < R} \Big[\log \phi(\alpha_i) - \log \sigma\Big],$$

where ϕ, Φ are the standard normal pdf and cdf. The classical Tobit is the special case $L = 0$, $R = +\infty$.

2.3 Olsen's reparameterisation

Optimisation is carried out in $(\gamma, \nu) = (\beta/\sigma,\ 1/\sigma)$. Olsen (1978) showed the resulting log-likelihood is **globally concave**, so any gradient-based optimiser converges to the unique maximum. After convergence the package back-transforms to (β, σ) and computes standard errors from the observed information matrix.

2.4 Truncated model

In a truncated sample, observations outside (L, R) are **absent** entirely. The density is the normal density renormalised by the probability of falling inside the interval:

$$f(y_i \mid L < Y_i^* < R) = \frac{\sigma^{-1}\,\phi\big((y_i - X_i'\beta)/\sigma\big)}{\Phi(\alpha_{R,i}) - \Phi(\alpha_{L,i})}.$$

2.5 Conditional means and marginal effects

Three conditional means are reported (Hansen, 2022, ch. 27):

- **Latent:** $\mathbb{E}[Y^* \mid X] = X'\beta$
- **Censored:** $\mathbb{E}[Y \mid X]$, accounting for the threshold mass
- **Truncated:** $\mathbb{E}[Y \mid X,\, L < Y < R]$

Marginal effects are available as **AME** (average over the sample) and **MEM** (evaluated at the mean regressor), each with delta-method standard errors. The censored marginal effect on $\mathbb{E}[Y \mid X]$ equals $\beta_j \cdot [\Phi(\alpha_R) - \Phi(\alpha_L)]$ — the latent slope shrunk by the probability of being interior.

2.6 Inference

Every fitted model carries an overall likelihood-ratio test against the intercept-only null model and McFadden's pseudo-R^2. Arbitrary nested models can be compared with `lr_test`, which returns $LR = 2(\ell_{\text{full}} - \ell_{\text{restricted}}) \sim \chi^2_q$.

3. Worked Examples

The remainder of this report reproduces the six example notebooks. Each is fully executed; the code and its output appear inline.

3.1 Example 1 — Two-Sided Censored Regression

In [1]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from censtrunc import CensoredRegression, lr_test

rng = np.random.default_rng(2026)
n = 3000
```

Data-generating process

$$Y_i^* = 1.0 + 0.5\,X_{1i} - 0.3\,X_{2i} + 0.2\,X_{3i} + e_i, \quad e_i \sim \mathcal{N}(0,1).$$

The observation rule clips values below $L = 0$ and above $R = 2.5$, producing a realistic two-sided censoring pattern (think 'satisfaction score on a 0-2.5 scale').

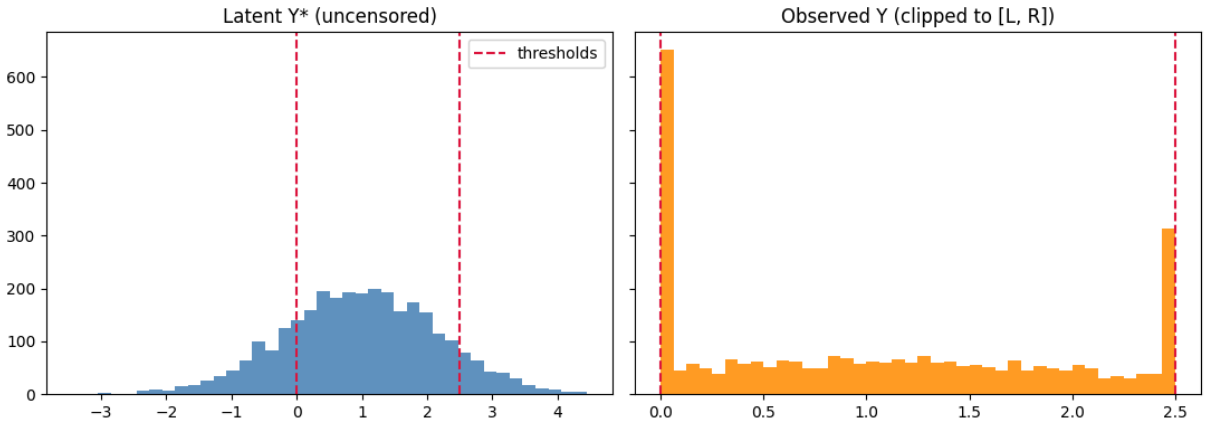
In [2]:
```
X = rng.normal(size=(n, 3))
beta_true = np.array([1.0, 0.5, -0.3, 0.2])
sigma_true = 1.0
y_star = beta_true[0] + X @ beta_true[1:] + rng.normal(scale=sigma_true, size=n)
L, R = 0.0, 2.5
y = np.clip(y_star, L, R)

print(f'Left-censored:   {np.mean(y == L):.1%}')
print(f'Right-censored:  {np.mean(y == R):.1%}')
print(f'Interior:        {np.mean((y > L) & (y < R)):.1%}')
```

```
Left-censored:   20.0%
Right-censored:  9.8%
Interior:        70.2%
```

Distribution of the latent and observed variables

The pile-ups at L and R in the observed distribution are the hallmark of two-sided censoring.

In [3]:
```
fig, axes = plt.subplots(1, 2, figsize=(11, 4), sharey=True)
axes[0].hist(y_star, bins=40, color='steelblue', alpha=0.85)
axes[0].set_title('Latent Y* (uncensored)')
axes[0].axvline(L, color='crimson', linestyle='--', label='thresholds')
axes[0].axvline(R, color='crimson', linestyle='--')
axes[0].legend()
axes[1].hist(y, bins=40, color='darkorange', alpha=0.85)
axes[1].set_title('Observed Y (clipped to [L, R])')
axes[1].axvline(L, color='crimson', linestyle='--')
axes[1].axvline(R, color='crimson', linestyle='--')
plt.tight_layout(); plt.show()
```



Fit the censored regression

The package's default optimisation uses Olsen's reparameterisation, which makes the log-likelihood globally concave and guarantees convergence of gradient-based optimisers (Olsen 1978).

In [4]:
```
model = CensoredRegression(left=L, right=R).fit(
    X, y, feature_names=['x1', 'x2', 'x3']
)
print(model.summary())
```

```
================================================================================
                          Censored Regression Results
================================================================================
Dep. Variable:           y                  No. Observations:        3000
Model:                   CensoredRegression Df Model:                4
Method:                  MLE                Df Residuals:            2995
Date:                    Tue, 02 Jun 2026   Log-Likelihood:          -3831.9928
Time:                    20:17:40           LL-Null:                 -4286.2006
AIC:                     7673.9855          LLR p-value:             1.323e-196
BIC:                     7704.0174          Pseudo R-squ.:           0.1060
Left threshold:          0                  Right threshold:         2.5
================================================================================
                     coef     std err         z      P>|z|      [0.025      0.975]
--------------------------------------------------------------------------------
sigma              0.9959      0.0165   60.3216     0.0000      0.9635      1.0282
const              0.9963      0.0190   52.4873     0.0000      0.9591      1.0336
x1                 0.5126      0.0195   26.2589     0.0000      0.4744      0.5509
x2                -0.2565      0.0188  -13.6315     0.0000     -0.2934     -0.2196
x3                 0.2030      0.0192   10.5893     0.0000      0.1655      0.2406
================================================================================
Left-censored observations:       601 (20.03%)
Right-censored observations:      293 ( 9.77%)
Uncensored observations:         2106 (70.20%)
Optimiser converged:             True
================================================================================
```

Compare against OLS

OLS on the censored sample is biased (Greene 1981): the slope coefficients are shrunk roughly in proportion to the censoring probability. Our Tobit-style MLE is consistent.

In [5]:
```python
X_with_int = np.column_stack([np.ones(n), X])
ols_beta, *_ = np.linalg.lstsq(X_with_int, y, rcond=None)

comparison = pd.DataFrame({
    'true':        beta_true,
    'OLS (biased)': ols_beta,
    'Censored MLE': model.coef_,
}, index=['const', 'x1', 'x2', 'x3'])
comparison['OLS bias']  = comparison['OLS (biased)']  - comparison['true']
comparison['MLE bias']  = comparison['Censored MLE']  - comparison['true']
comparison.round(4)
```

Out[5]:

	true	OLS (biased)	Censored MLE	OLS bias	MLE bias
const	1.0	1.0706	0.9963	0.0706	-0.0037
x1	0.5	0.3648	0.5126	-0.1352	0.0126
x2	-0.3	-0.1794	-0.2565	0.1206	0.0435
x3	0.2	0.1460	0.2030	-0.0540	0.0030

Predictions: six quantities, one call

`model.predict(X)` returns a dataframe with all six quantities — three conditional means and three region probabilities. Each column has a one-letter mnemonic:

- `h` — *hidden* / latent mean $\mathbb{E}[Y^* \mid X] = X'\beta$
- `c` — *censored* mean $\mathbb{E}[Y \mid X]$
- `t` — *truncated* mean $\mathbb{E}[Y \mid X, L < Y < R]$

- `l` — left-region probability $P(Y = L \mid X) = \Phi(\alpha_L)$
- `m` — middle probability $P(L < Y < R \mid X) = \Phi(\alpha_R) - \Phi(\alpha_L)$
- `r` — right-region probability $P(Y = R \mid X) = 1 - \Phi(\alpha_R)$

In [6]:
```
X_show = X[:6]
model.predict(X_show).round(3)   # default = all six columns
```

Out[6]:

	latent	censored	truncated	prob_left	prob_interior	prob_right
0	0.143	0.470	0.816	0.443	0.548	0.009
1	1.489	1.438	1.351	0.067	0.778	0.155
2	0.704	0.831	1.023	0.240	0.725	0.036
3	0.800	0.901	1.062	0.211	0.745	0.044
4	1.018	1.068	1.152	0.153	0.778	0.068
5	0.896	0.974	1.101	0.184	0.762	0.054

Subsets are selected with the same letter codes. For example, just the three probabilities (note that each row sums to one) or just the hidden mean (a 1-D array):

In [7]:
```
model.predict(X_show, kind='lmr').round(3)
```

Out[7]:

	prob_left	prob_interior	prob_right
0	0.443	0.548	0.009
1	0.067	0.778	0.155
2	0.240	0.725	0.036
3	0.211	0.745	0.044
4	0.153	0.778	0.068
5	0.184	0.762	0.054

In [8]:
```
model.predict(X_show, kind='h')          # single letter -> 1-D ndarray
```

Out[8]:
```
array([0.14305362, 1.48881203, 0.70415823, 0.80034359, 1.01807358,
       0.89647868])
```

Long names — `kind='latent'`, `'censored'`, `'truncated'` — remain valid and return 1-D arrays for backward compatibility.

Marginal effects via `get_margeff`

The marginal-effects API mirrors statsmodels' `get_margeff`: one method, with options for *where* to evaluate (`at`) and *what* to report (`method`). The summary prints in the familiar statsmodels layout.

In [9]:
```
# AME = average over the sample (at='overall'); statsmodels-style summary
print(model.get_margeff(at='overall', method='dydx', kind='censored').summary())
```

```
                    Censored Regression Marginal Effects
====================================
Dep. Variable:                      y
Method:                          dydx
At:                           overall
==============================================================================
                dy/dx    std err          z      P>|z|      [0.025      0.975]
------------------------------------------------------------------------------
x1             0.3600      0.012     30.227      0.000       0.337       0.383
x2            -0.1801      0.013    -14.113      0.000      -0.205      -0.155
x3             0.1426      0.013     10.805      0.000       0.117       0.168
==============================================================================
```

`at='overall'` is the AME; `at='mean'` is the marginal effect at the mean (MEM). Below we compare the censored marginal effect across evaluation points and against the latent effect (which equals β exactly).

In [10]:
```
import pandas as pd
summary = pd.DataFrame({
    'latent (=beta)':   model.get_margeff(at='overall', kind='latent').margeff,
    'censored AME':     model.get_margeff(at='overall', kind='censored').margeff,
    'censored MEM':     model.get_margeff(at='mean',    kind='censored').margeff,
    'truncated AME':    model.get_margeff(at='overall', kind='truncated').margeff,
}, index=['x1', 'x2', 'x3'])
summary.round(4)
```

Out[10]:

	latent (=beta)	censored AME	censored MEM	truncated AME
x1	0.5126	0.3600	0.3973	0.2005
x2	-0.2565	-0.1801	-0.1988	-0.1003
x3	0.2030	0.1426	0.1573	0.0794

`method` switches between the derivative and elasticities: `dydx` (derivative), `eyex` (elasticity), `dyex` / `eydx` (semi-elasticities). Discrete regressors can be handled with `dummy=True` (a 0->1 difference rather than a derivative).

In [11]:
```
elas = model.get_margeff(at='overall', method='eyex', kind='censored')
elas.summary_frame().round(4)
```

Out[11]:

	eyex	std err	z	P>|z|	[0.025	0.975]
x1	-0.1238	0.0093	-13.3389	0.0	-0.1420	-0.1056
x2	-0.0317	0.0045	-7.0004	0.0	-0.0406	-0.0228
x3	-0.0193	0.0035	-5.4717	0.0	-0.0263	-0.0124

Marginal effects on the region probabilities

Beyond the conditional mean, we can ask how a regressor shifts the *probability* of each region: landing at the left threshold ($P(Y = L) = \Phi(\alpha_L)$), in the interior ($P(L < Y < R) = \Phi(\alpha_R) - \Phi(\alpha_L)$), or at the right threshold (

$P(Y = R) = 1 - \Phi(\alpha_R)$). These are selected with `kind='prob-left'`, `'prob-interior'`, `'prob-right'`. Because the three probabilities sum to one, their marginal effects sum to zero.

In [12]:
```
prob_effects = pd.DataFrame({
    'P(Y=L)':     model.get_margeff(kind='prob-left').margeff,
    'P(L<Y<R)':   model.get_margeff(kind='prob-interior').margeff,
    'P(Y=R)':     model.get_margeff(kind='prob-right').margeff,
}, index=['x1', 'x2', 'x3'])
prob_effects['sum (=0)'] = prob_effects.sum(axis=1)
prob_effects.round(4)
```

Out[12]:

	P(Y=L)	P(L<Y<R)	P(Y=R)	sum (=0)
x1	-0.1221	0.0461	0.0760	0.0
x2	0.0611	-0.0231	-0.0380	0.0
x3	-0.0484	0.0183	0.0301	0.0

A positive coefficient pushes observations out of the left pile-up and toward the right one, so the `P(Y=L)` effect is negative and the `P(Y=R)` effect is positive; the `sum (=0)` column confirms the adding-up constraint holds numerically.

Likelihood-ratio test

We test whether `x3` is jointly redundant by fitting a restricted model that omits it, then comparing log-likelihoods.

In [13]:
```
restricted = CensoredRegression(left=L, right=R).fit(X[:, :2], y)
res = lr_test(model, restricted)
print(res)
```

```
Likelihood-ratio test
  LR statistic : 111.3544
  df           : 1
  p-value      : 4.949e-26
  log-lik full : -3831.9928
  log-lik rest.: -3887.6700
  n            : 3000
```

If the p-value is small, we reject the restriction (drop `x3`). Here `x3` has a true coefficient of 0.2 and a large sample, so the test correctly rejects.

LR test from a hypothesis string

Re-fitting a restricted model by hand is fine for simple drop-a-column cases, but cumbersome for joint or composite restrictions. For those, `model.lr_test(hypotheses)` mirrors statsmodels' `f_test` and accepts linear restrictions directly as a string — single, joint, or with arithmetic on parameter names.

In [14]:
```
# Drop a single regressor (same null as the two-model test above)
print(model.lr_test('x3 = 0'))
```

```
Likelihood-ratio test
  LR statistic : 111.3544
  df           : 1
  p-value      : 4.949e-26
  log-lik full : -3831.9928
  log-lik rest.: -3887.6700
  n            : 3000
```

In [15]:
```
# Joint restriction: x2 AND x3 are zero (composite null)
print(model.lr_test('(x2 = 0), (x3 = 0)'))
```

```
Likelihood-ratio test
  LR statistic : 286.4290
  df           : 2
  p-value      : 6.35e-63
  log-lik full : -3831.9928
  log-lik rest.: -3975.2073
  n            : 3000
```

In [16]:
```
# Composite restriction with arithmetic: x1 - 2*x2 = 0
# (equivalent to testing whether x1 equals twice x2)
print(model.lr_test('x1 - 2*x2 = 0'))
```

```
Likelihood-ratio test
  LR statistic : 549.6200
  df           : 1
  p-value      : 1.523e-121
  log-lik full : -3831.9928
  log-lik rest.: -4106.8028
  n            : 3000
```

Each call refits the model under the linear constraint $R\hat{\theta} = r$ (via `scipy.optimize` with `LinearConstraint`) and reports the asymptotic LR statistic $2(\ell_{\text{full}} - \ell_{\text{rest.}}) \sim \chi^2_q$.

3.2 Example 2 — Classical Tobit (Fair Affairs Data)

In [17]:
```
import numpy as np
import pandas as pd
import statsmodels.api as sm
from censtrunc import CensoredRegression

data = sm.datasets.fair.load_pandas().data
print('n =', len(data))
print(f"share with affairs == 0: {(data['affairs'] == 0).mean():.1%}")
data.head()
```

```
n = 6366
share with affairs == 0: 67.8%
```

Out[17]:

	rate_marriage	age	yrs_married	children	religious	educ	occupation	occupatio
0	3.0	32.0	9.0	3.0	3.0	17.0	2.0	
1	3.0	27.0	13.0	3.0	1.0	14.0	3.0	
2	4.0	22.0	2.5	0.0	1.0	16.0	3.0	
3	4.0	37.0	16.5	4.0	3.0	16.0	5.0	
4	5.0	27.0	9.0	1.0	1.0	14.0	3.0	

Specification

Following Fair (1978), we regress the count of affairs on the standard set of demographic and marital variables.

In [18]:
```
covariates = ['rate_marriage', 'age', 'yrs_married', 'children',
              'religious', 'educ', 'occupation', 'occupation_husb']
X = data[covariates]
y = data['affairs']
```

Fit

We tell `CensoredRegression` that the left threshold is zero; the right threshold is left unspecified.

In [19]:
```
model = CensoredRegression(left=0.0).fit(X, y)
print(model.summary())
```

```
================================================================================
                          Censored Regression Results
================================================================================
Dep. Variable:          y                  No. Observations:        6366
Model:                  CensoredRegression Df Model:                9
Method:                 MLE                Df Residuals:            6356
Date:                   Tue, 02 Jun 2026   Log-Likelihood:          -7804.3803
Time:                   20:17:41           LL-Null:                 -8155.0240
AIC:                    15628.7605         LLR p-value:             3.780e-146
BIC:                    15696.3478         Pseudo R-squ.:           0.0430
Left threshold:         0                  Right threshold:         none
================================================================================
                    coef     std err         z     P>|z|      [0.025      0.975]
--------------------------------------------------------------------------------
sigma               4.4990      0.0771   58.3463    0.0000      4.3478      4.6501
const               7.8354      0.7135   10.9818    0.0000      6.4370      9.2338
rate_marriage      -1.5313      0.0734  -20.8514    0.0000     -1.6752     -1.3873
age                -0.1051      0.0248   -4.2393    0.0000     -0.1537     -0.0565
yrs_married         0.1282      0.0264    4.8576    0.0000      0.0765      0.1799
children           -0.0276      0.0771   -0.3576    0.7206     -0.1788      0.1236
religious          -0.9434      0.0849  -11.1077    0.0000     -1.1099     -0.7769
educ               -0.0857      0.0376   -2.2772    0.0228     -0.1594     -0.0119
occupation          0.3125      0.0819    3.8150    0.0001      0.1520      0.4731
occupation_husb     0.0143      0.0555    0.2578    0.7966     -0.0944      0.1230
================================================================================
Left-censored observations:      4313 (67.75%)
Right-censored observations:        0 ( 0.00%)
Uncensored observations:         2053 (32.25%)
Optimiser converged:             True
================================================================================
```

Marginal effects

Because the response is left-censored at zero, the censored marginal effects are smaller in absolute value than the latent ones. The latent (uncensored) coefficients describe the propensity scale, while the censored marginal effects describe the *expected number* of affairs.

In [20]:
```
ame_cens = model.ame(kind='censored').to_dataframe()
ame_lat  = model.ame(kind='latent').to_dataframe()
pd.concat([
```

```
    ame_lat[['dy/dx']].rename(columns={'dy/dx': 'latent'}),
    ame_cens[['dy/dx', 'std err', 'P>|z|']].rename(columns={'dy/dx': 'censored E[Y]'}),
], axis=1).round(4)
```

Out[20]:

	latent	censored E[Y]	std err	P>|z|
rate_marriage	-1.5313	-0.4326	0.0213	0.0000
age	-0.1051	-0.0297	0.0070	0.0000
yrs_married	0.1282	0.0362	0.0074	0.0000
children	-0.0276	-0.0078	0.0218	0.7206
religious	-0.9434	-0.2665	0.0243	0.0000
educ	-0.0857	-0.0242	0.0106	0.0228
occupation	0.3125	0.0883	0.0232	0.0001
occupation_husb	0.0143	0.0040	0.0157	0.7966

Sanity check vs. OLS

Naive OLS on the censored sample understates the slope coefficients (Greene 1981). The Tobit MLE corrects this.

In [21]:

```
X_int = sm.add_constant(X)
ols = sm.OLS(y, X_int).fit()
pd.DataFrame({
    'OLS coef':    ols.params.values,
    'Tobit coef':  model.coef_,
}, index=ols.params.index).round(4)
```

Out[21]:

	OLS coef	Tobit coef
const	3.6235	7.8354
rate_marriage	-0.4205	-1.5313
age	-0.0146	-0.1051
yrs_married	-0.0160	0.1282
children	-0.0171	-0.0276
religious	-0.2437	-0.9434
educ	-0.0174	-0.0857
occupation	0.0658	0.3125
occupation_husb	0.0040	0.0143

3.3 Example 3 — Truncated Regression

In [22]:
```
import numpy as np
import pandas as pd
from censtrunc import TruncatedRegression

rng = np.random.default_rng(7)
n_population = 8000
```

Simulate a truncated dataset

We draw a large population, then keep only observations with $0 < Y^* < 2.5$ — about 60% of the population survives the double-sided truncation in this DGP.

In [23]:
```
X_pop = rng.normal(size=(n_population, 2))
beta_true = np.array([1.0, 0.5, -0.3])
y_star = beta_true[0] + X_pop @ beta_true[1:] + rng.normal(scale=1.0, size=n_population)
L, R = 0.0, 2.5
mask = (y_star > L) & (y_star < R)
X = X_pop[mask]
y = y_star[mask]
print(f'Observed after truncation: {len(y)} / {n_population}')
```

```
Observed after truncation: 5741 / 8000
```

Fit the truncated regression

In [24]:
```
model = TruncatedRegression(left=L, right=R).fit(
    X, y, feature_names=['x1', 'x2']
)
print(model.summary())
```

```
================================================================================
                          Truncated Regression Results
================================================================================
Dep. Variable:          y                  No. Observations:        5741
Model:                  TruncatedRegression Df Model:               3
Method:                 MLE                Df Residuals:            5737
Date:                   Tue, 02 Jun 2026   Log-Likelihood:          -4830.9612
Time:                   20:17:41           LL-Null:                 -5136.1192
AIC:                    9669.9225          LLR p-value:             2.962e-133
BIC:                    9696.5440          Pseudo R-squ.:           0.0594
Left truncation:        0                  Right truncation:        2.5
================================================================================
                   coef     std err         z     P>|z|      [0.025      0.975]
--------------------------------------------------------------------------------
sigma              0.9944      0.0316   31.5070    0.0000      0.9326      1.0563
const              0.9946      0.0245   40.6756    0.0000      0.9467      1.0425
x1                 0.4881      0.0335   14.5918    0.0000      0.4226      0.5537
x2                -0.3105      0.0268  -11.6040    0.0000     -0.3629     -0.2580
================================================================================
Optimiser converged:          True
================================================================================
```

Comparison: OLS on the truncated sample is biased

Naive OLS on the truncated sample treats the data as if it came from an unrestricted population — leading to biased slope estimates, typically shrunk toward zero.

In [25]:
```
X_int = np.column_stack([np.ones(len(y)), X])
ols_beta, *_ = np.linalg.lstsq(X_int, y, rcond=None)

comparison = pd.DataFrame({
    'true':         beta_true,
    'OLS (biased)': ols_beta,
    'Truncated MLE': model.coef_,
}, index=['const', 'x1', 'x2'])
comparison['OLS bias']           = comparison['OLS (biased)']  - comparison['true']
comparison['Truncated MLE bias'] = comparison['Truncated MLE'] - comparison['true']
comparison.round(4)
```

Out[25]:

	true	OLS (biased)	Truncated MLE	OLS bias	Truncated MLE bias
const	1.0	1.1475	0.9946	0.1475	-0.0054
x1	0.5	0.1952	0.4881	-0.3048	-0.0119
x2	-0.3	-0.1241	-0.3105	0.1759	-0.0105

Predictions

For the truncated model only two prediction quantities are meaningful (there is no mass at the thresholds, so the region-probability columns do not apply):

- `h` (`latent`): $X'\hat{\beta}$ — the population mean
- `t` (`truncated`): $\mathbb{E}[Y|X, L < Y < R]$

In [26]:
```
preds = model.predict(X[:6])      # default = 'ht' -> both columns
preds.insert(0, 'observed', y[:6])
preds.round(3)
```

Out[26]:

	observed	latent	truncated
0	1.590	0.902	1.104
1	0.317	1.137	1.202
2	1.514	1.081	1.178
3	1.376	0.947	1.122
4	1.085	0.764	1.047
5	0.074	0.481	0.936

3.4 Example 4 — Validation against OLS and R

In [27]:
```
import numpy as np
import pandas as pd
import statsmodels.api as sm
from censtrunc import CensoredRegression, TruncatedRegression

rng = np.random.default_rng(0)
n = 1000
X = rng.normal(size=(n, 3))
y = 2.0 + 1.5*X[:,0] - 0.7*X[:,1] + 0.3*X[:,2] + rng.normal(scale=1.3, size=n)
```

No censoring ⇒ OLS

We set `left=-1e6`, `right=1e6` so that no observation is censored. The censored log-likelihood then reduces to the Gaussian log-likelihood, whose maximiser is exactly the OLS coefficient vector.

In [28]:
```
ols = sm.OLS(y, sm.add_constant(X)).fit()
cens = CensoredRegression(left=-1e6, right=1e6).fit(X, y)
trunc = TruncatedRegression(left=-1e6, right=1e6).fit(X, y)

pd.DataFrame({
    'OLS':            np.asarray(ols.params),
    'Censored MLE':   cens.coef_,
    'Truncated MLE':  trunc.coef_,
}, index=['const', 'x1', 'x2', 'x3']).round(6)
```

Out[28]:

	OLS	Censored MLE	Truncated MLE
const	2.046512	2.046482	2.046512
x1	1.434191	1.434221	1.434191
x2	-0.626598	-0.626603	-0.626598
x3	0.345980	0.345982	0.345980

The columns agree to several decimals. We can quantify the maximum discrepancy and confirm the scale parameter and log-likelihood match too (using the MLE scale $\hat{\sigma} = \sqrt{\mathrm{SSR}/n}$).

In [29]:
```
ols_beta = np.asarray(ols.params)
print(f'max |beta_censored - beta_OLS|  = {np.max(np.abs(cens.coef_ - ols_beta)):.2e}')
print(f'max |beta_truncated - beta_OLS| = {np.max(np.abs(trunc.coef_ - ols_beta)):.2e}')
print(f'|sigma_censored - sqrt(SSR/n)|  = {abs(cens.sigma_ - np.sqrt(ols.ssr/n)):.2e}')
print(f'|loglik_censored - loglik_OLS|  = {abs(cens.llf_ - ols.llf):.2e}')
```

```
max |beta_censored - beta_OLS|  = 3.00e-05
max |beta_truncated - beta_OLS| = 4.44e-16
|sigma_censored - sqrt(SSR/n)|  = 7.55e-07
|loglik_censored - loglik_OLS|  = 5.90e-07
```

All discrepancies are at the level of the optimiser tolerance. The estimator reduces to OLS in the no-censoring limit, as it should, so the likelihood and its optimisation are wired up correctly.

With censoring: agreement with R, disagreement with OLS

The decisive test is a genuinely censored dataset. We compare four numbers per coefficient:

- **true** — the data-generating value;
- **censtrunc** — this package's MLE;
- **R survreg** — R's Gaussian `survreg`, the engine behind `AER::tobit`, computed by an entirely independent codebase;
- **OLS** — naive least squares, which is biased under censoring.

Two correct maximum-likelihood implementations must converge to the *same unique* optimum, so `censtrunc` and R should agree to optimiser tolerance — while both should differ from the biased OLS and sit close to the truth.

In [30]:
```
import shutil, subprocess, json, tempfile, os
from pathlib import Path

rng2 = np.random.default_rng(20260527)
nc = 4000
Xc = rng2.normal(size=(nc, 2))
y_star_c = 1.0 + 0.7*Xc[:, 0] - 0.4*Xc[:, 1] + rng2.normal(size=nc)
Lc, Rc = 0.0, 2.5
yc = np.clip(y_star_c, Lc, Rc)

cens = CensoredRegression(left=Lc, right=Rc).fit(Xc, yc)
ols_c = np.asarray(sm.OLS(yc, sm.add_constant(Xc)).fit().params)

# Run R's survreg via the bundled reference script, if R is available.
def _find_r_script():
    for c in [Path('tests/reference/fit_tobit.R'),
              Path('../tests/reference/fit_tobit.R')]:
        if c.exists():
            return c
    return None

r_beta = r_sigma = None
rscript, script_path = shutil.which('Rscript'), _find_r_script()
if rscript and script_path:
    fd, csvp = tempfile.mkstemp(suffix='.csv'); os.close(fd)
    np.savetxt(csvp, np.column_stack([yc, Xc]), delimiter=',', header='y,x1,x2', comments='')
    try:
        out = subprocess.run([rscript, str(script_path), csvp, str(Lc), str(Rc)],
                             capture_output=True, text=True, timeout=120, check=True)
        rt = json.loads(out.stdout)['tobit']
        r_beta = np.array([rt['coef']['(Intercept)'], rt['coef']['x1'], rt['coef']['x2']])
        r_sigma = float(rt['scale'])
    except Exception:
        r_beta = None
    finally:
        os.remove(csvp)
```

In [31]:
```
if r_beta is not None:
    table = pd.DataFrame({
        'true':         [1.0, 0.7, -0.4],
        'censtrunc':    cens.coef_,
        'R survreg':    r_beta,
        'OLS (biased)': ols_c,
```

```
    }, index=['const', 'x1', 'x2'])
    display(table.round(6))
    print(f'max |censtrunc - R|  = {np.max(np.abs(cens.coef_ - r_beta)):.2e}  (independent so
    print(f'|sigma_censtrunc - sigma_R| = {abs(cens.sigma_ - r_sigma):.2e}')
    print(f'max |censtrunc - OLS| = {np.max(np.abs(cens.coef_ - ols_c)):.3f}  (Tobit correcti
else:
    print('R not available at build time.')
    print('The live comparison runs in the test suite: tests/test_r_reference.py')
    print('censtrunc estimates:', cens.coef_.round(4))
    print('OLS (biased):       ', ols_c.round(4))
```

	true	censtrunc	R survreg	OLS (biased)
const	1.0	1.014284	1.014284	1.090364
x1	0.7	0.701331	0.701331	0.470177
x2	-0.4	-0.410105	-0.410105	-0.277838

```
max |censtrunc - R|  = 3.05e-08  (independent solvers, same optimum)
|sigma_censtrunc - sigma_R| = 2.41e-08
max |censtrunc - OLS| = 0.231  (Tobit correction is real)
```

The `censtrunc` and R columns are identical to about six decimals, yet `max |censtrunc - R|` is a tiny *non-zero* number (~1e-8): the two independent optimisers land on the same maximum without being bit-for-bit copies. Both recover the true coefficients, while OLS is visibly biased toward zero — exactly the censoring attenuation predicted by Greene (1981).

Probit limit: tight thresholds reduce Tobit to probit

Consider the censored regression with $L = R = c$. With no interior region the likelihood reduces to

$$\ell(\beta, \sigma) = \sum_i \mathbf{1}\{y_i = c\} \log \Phi\left(\tfrac{c - X_i'\beta}{\sigma}\right) + \mathbf{1}\{y_i > c\} \log\left[1 - \Phi\left(\tfrac{c - X_i'\beta}{\sigma}\right)\right],$$

which is *exactly* the probit log-likelihood for the indicator $D_i = \mathbf{1}\{Y_i^* > c\}$, identifying $\beta_{\text{probit}} = \beta_{\text{tobit}} / \sigma_{\text{tobit}}$ (Hansen 2022, §27.4).

We cannot set $L = R$ literally (then σ is unidentified), but a *narrow band* $[c - \varepsilon, c + \varepsilon]$ is close enough. A handful of interior observations identifies σ; the rest of the data act like a binary outcome. So as $\varepsilon \to 0$ we expect $\hat\beta_{\text{tobit}} / \hat\sigma_{\text{tobit}} \to \hat\beta_{\text{probit}}$.

In [32]:
```
from statsmodels.discrete.discrete_model import Probit

rng3 = np.random.default_rng(20250601)
n3 = 8000
Xp = rng3.normal(size=(n3, 2))
beta_p = np.array([0.4, 1.2, -0.8])
y_star_p = beta_p[0] + Xp @ beta_p[1:] + rng3.normal(size=n3)
y_bin = (y_star_p > 0.0).astype(float)
probit = Probit(y_bin, sm.add_constant(Xp)).fit(disp=False)

def fit_band(eps: float):
    y_obs = np.clip(y_star_p, -eps, eps)
    m = CensoredRegression(left=-eps, right=eps).fit(Xp, y_obs)
```

```
    return m.coef_ / m.sigma_   # the ratio that should match probit

table = pd.DataFrame({
    'probit beta':                  np.asarray(probit.params),
    'tobit / sigma  (eps=0.5)':     fit_band(0.50),
    'tobit / sigma  (eps=0.1)':     fit_band(0.10),
    'tobit / sigma  (eps=0.05)':    fit_band(0.05),
}, index=['const', 'x1', 'x2'])
table.round(4)
```

Out[32]:

	probit beta	tobit / sigma (eps=0.5)	tobit / sigma (eps=0.1)	tobit / sigma (eps=0.05)
const	0.3791	0.3755	0.3728	0.3744
x1	1.2218	1.1923	1.2134	1.2192
x2	-0.7774	-0.7726	-0.7754	-0.7783

As the band shrinks, the tobit-to-probit ratio approaches the probit estimate. The intercept drifts more than the slopes — a finite-band effect that scales like ε/σ — but at $\varepsilon = 0.05$ the slopes agree to within a few percent, which checks the boundary terms of ℓ separately from the OLS limit above.

3.5 Example 5 — Visual: Truncated vs Censored

In [33]:
```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
from censtrunc import CensoredRegression, TruncatedRegression
from censtrunc._means import value_for_kind
from censtrunc._utils import _prepare_design_matrix

rng = np.random.default_rng(2026)
n = 250
x = rng.uniform(-10, 10, size=n)
sigma_true = 2.0
y_star = 1.0 * x + rng.normal(scale=sigma_true, size=n)
L, R = -5.0, 5.0
```

Truncated dataset: observations with $y \notin (L, R)$ are dropped entirely. **Censored dataset:** they are clipped to the nearest threshold. The latent linear function $y = x$ is the same in both.

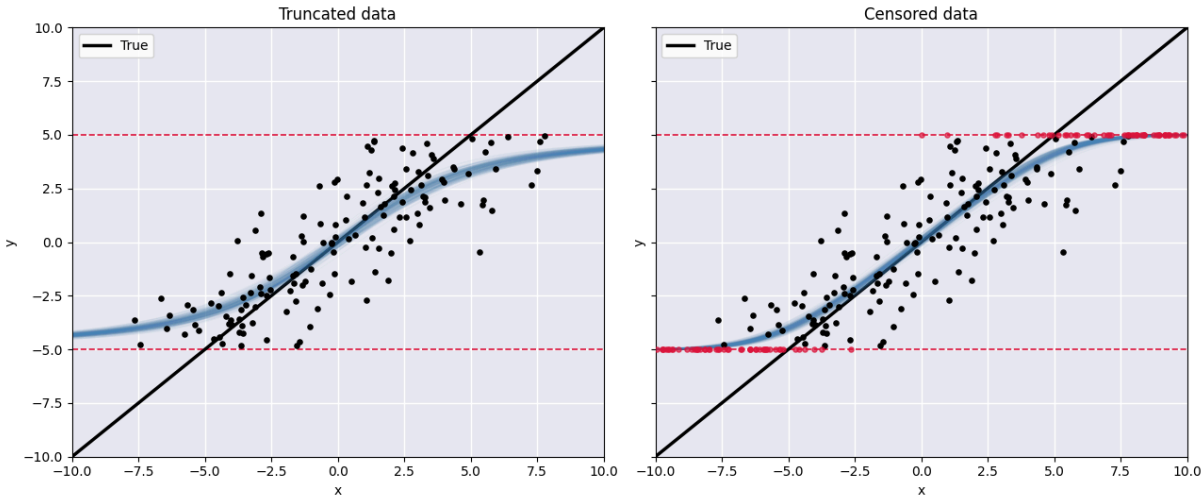
In [34]:
```
mask = (y_star > L) & (y_star < R)
x_t, y_t = x[mask], y_star[mask]
x_c, y_c = x.copy(), np.clip(y_star, L, R)

mt = TruncatedRegression(left=L, right=R).fit(x_t.reshape(-1, 1), y_t)
mc = CensoredRegression(left=L, right=R).fit(x_c.reshape(-1, 1), y_c)
print(f'truncated:  n={len(y_t)}, beta_hat = {mt.coef_.round(3)}')
print(f'censored:   n={len(y_c)}, beta_hat = {mc.coef_.round(3)}')
```

```
truncated:  n=146, beta_hat = [-0.003  0.946]
censored:   n=250, beta_hat = [0.059 0.939]
```

Now we build a prediction band from asymptotic uncertainty. We draw 200 parameter vectors from the asymptotic normal distribution of $\hat{\theta}$, compute the corresponding conditional mean curve (truncated on the left, censored on the right), and plot them with low opacity. The pile-up at the censoring thresholds is shown in red on the right.

In [35]:
```
def prediction_band(model, x_grid, kind, n_draws=200, seed=0):
    rng = np.random.default_rng(seed)
    samples = rng.multivariate_normal(model.params_, model.cov_params_, size=n_draws)
    samples = samples[samples[:, 0] > 0]   # keep only feasible sigma > 0
    Xg, _ = _prepare_design_matrix(x_grid.reshape(-1, 1), fit_intercept=True)
    curves = np.empty((samples.shape[0], x_grid.size))
    for i, theta in enumerate(samples):
        sigma, beta = theta[0], theta[1:]
        curves[i] = value_for_kind(
            beta, sigma, Xg,
            model._left, model._right, model._has_left, model._has_right, kind,
        )
    return curves

x_grid = np.linspace(-10, 10, 300)
curves_t = prediction_band(mt, x_grid, kind='truncated')
curves_c = prediction_band(mc, x_grid, kind='censored')
```

In [36]:
```
fig, axes = plt.subplots(1, 2, figsize=(13, 5.5), sharey=True)
for ax in axes:
    ax.set_facecolor('#eaeaf2')
    ax.grid(True, color='white', linewidth=1)
    ax.axhline(L, color='crimson', linestyle='--', linewidth=1.2)
    ax.axhline(R, color='crimson', linestyle='--', linewidth=1.2)
    ax.set_xlim(-10, 10); ax.set_ylim(-10, 10)
    ax.set_xlabel('x'); ax.set_ylabel('y')
    ax.plot(x_grid, x_grid, color='black', linewidth=2.5, label='True')

# Left panel: truncated
ax = axes[0]
ax.set_title('Truncated data')
for c in curves_t:
    ax.plot(x_grid, c, color='steelblue', alpha=0.04, linewidth=1)
ax.scatter(x_t, y_t, c='black', s=14, zorder=5)
ax.legend(loc='upper left')

# Right panel: censored
ax = axes[1]
ax.set_title('Censored data')
for c in curves_c:
    ax.plot(x_grid, c, color='steelblue', alpha=0.04, linewidth=1)
is_at_bound = (y_c == L) | (y_c == R)
ax.scatter(x_c[~is_at_bound], y_c[~is_at_bound], c='black', s=14, zorder=5)
ax.scatter(x_c[is_at_bound],  y_c[is_at_bound],  c='crimson', s=14, zorder=5, alpha=0.7)
ax.legend(loc='upper left')

plt.tight_layout(); plt.show()
```



What to look for.

- The black line is the true linear relation $y^* = x$.
- The blue band is the model's MLE prediction with asymptotic uncertainty; on the **left** it bends inside (L, R) because the truncated mean $\mathbb{E}[Y \mid X, L < Y < R]$ is shrunk toward the interval centre; on the **right** it flattens near each threshold because the censored mean $\mathbb{E}[Y \mid X]$ mixes the latent linear part with the threshold mass.
- Red dots on the right panel are the censored pile-ups at $y = L$ and $y = R$. The truncated dataset has no such pile-ups because those observations are absent entirely.
- Both fitted curves are close to the true line in the interior, where the data are most informative; both lose precision near and beyond the thresholds, but neither shows the strong attenuation toward zero that OLS would suffer.

3.6 Example 6 — Heckman Selection (Heckit)

In [37]:
```
import numpy as np
import pandas as pd
import statsmodels.api as sm
from censtrunc import HeckitRegression

rng = np.random.default_rng(42)
n = 4000
# Shared regressor (in both equations); two exclusive regressors
shared  = rng.normal(size=n)
x_only  = rng.normal(size=n)       # in the outcome eq. only
z_only  = rng.normal(size=n)       # in the selection eq. only (exclusion)

beta_true  = np.array([1.0,  0.5, -0.3])     # const, shared, x_only
gamma_true = np.array([0.0,  0.3,  0.6])     # const, shared, z_only
rho_true, sigma_true = 0.6, 1.0

Sigma = np.array([[sigma_true**2, rho_true*sigma_true],
                  [rho_true*sigma_true, 1.0]])
errs = rng.multivariate_normal([0.0, 0.0], Sigma, size=n)
e, u = errs[:, 0], errs[:, 1]
```

```
X = np.column_stack([shared, x_only])
Z = np.column_stack([shared, z_only])
S = (gamma_true[0] + Z @ gamma_true[1:] + u > 0).astype(int)
y_full = beta_true[0] + X @ beta_true[1:] + e
y = np.where(S == 1, y_full, np.nan)
print(f'Selected (S=1): {S.sum()} / {n}  ({S.mean():.1%})')
```

```
Selected (S=1): 2022 / 4000  (50.5%)
```

Why OLS on the selected sample is biased

Because u and e are correlated ($\rho > 0$), individuals with high e tend to have high u and therefore are more likely to be selected. The conditional mean for the selected sample is

$$\mathbb{E}[Y \mid X, S = 1] = X'\beta + \rho\sigma\,\lambda(Z'\gamma),$$

with $\lambda(\cdot)$ the inverse Mills ratio. Omitting $\lambda(Z'\gamma)$ from the regression — what naive OLS does — produces omitted-variable bias on β.

In [38]:
```
sel = ~np.isnan(y)
X_full = sm.add_constant(X)
ols = sm.OLS(y[sel], X_full[sel]).fit()
print('OLS on selected subsample (biased):')
print(np.asarray(ols.params).round(4))
print('truth:', beta_true)
```

```
OLS on selected subsample (biased):
[ 1.3651  0.4366 -0.3037]
truth: [ 1.   0.5 -0.3]
```

Two-step Heckit

Heckman's two-step recipe:

1. Probit of S on Z to obtain $\hat{\gamma}$.
2. Compute the inverse Mills ratio $\hat{\lambda}_i = \phi(Z_i'\hat{\gamma}) / \Phi(Z_i'\hat{\gamma})$ for selected observations.
3. OLS of y_i on $(X_i, \hat{\lambda}_i)$ for selected i. The coefficients are $\hat{\beta}$ and $\hat{\rho}\hat{\sigma}$.

In [39]:
```
m_two = HeckitRegression(method='twostep').fit(y, X, Z)
print(m_two.summary())
```

```
                        Heckman Selection Regression
==============================================================================
Method:                   twostep                No. Observations:        4000
Selected (S=1):           2022                   Censored (S=0):          1978
sigma_e:                  0.952857               rho:                     0.531381
==============================================================================
                      Outcome equation (Y* = X * beta + e)
------------------------------------------------------------------------------
                     coef     std err         z     P>|z|      [0.025    0.975]
------------------------------------------------------------------------------
const              1.0267      0.0491   20.9278    0.0000      0.9306    1.1229
x1                 0.4903      0.0223   21.9471    0.0000      0.4465    0.5341
x2                -0.3010      0.0209  -14.4109    0.0000     -0.3419   -0.2601
------------------------------------------------------------------------------
             Selection equation (S* = Z * gamma + u, var(u) = 1)
------------------------------------------------------------------------------
                     coef     std err         z     P>|z|      [0.025    0.975]
------------------------------------------------------------------------------
const              0.0333      0.0213    1.5611    0.1185     -0.0085    0.0751
x1                 0.2585      0.0218   11.8357    0.0000      0.2157    0.3013
x2                 0.6260      0.0242   25.8990    0.0000      0.5786    0.6734
==============================================================================
Note: two-step second-stage standard errors are naive; call .bootstrap(y, X, Z) for proper Heckman-correc
==============================================================================
```

Joint MLE Heckit

Maximise the full log-likelihood (Hansen 2022, §27.10):

$$\ell = \sum_{S_i=0} \log[1 - \Phi(Z_i'\gamma)] + \sum_{S_i=1} \left\{ \log \Phi\left(\frac{Z_i'\gamma + (\rho/\sigma)(Y_i - X_i'\beta)}{\sqrt{1-\rho^2}} \right) - \tfrac{1}{2}\log(2\pi\sigma^2) - \frac{(}{}$$

Asymptotically efficient, somewhat slower than the two-step.

In [40]:
```
m_ml = HeckitRegression(method='mle').fit(y, X, Z)
print(m_ml.summary())
```

```
                        Heckman Selection Regression
==============================================================================
Method:                      mle                 No. Observations:        4000
Selected (S=1):           2022                   Censored (S=0):          1978
sigma_e:                  0.958607               rho:                     0.552343
Log-Likelihood:          -4924.8374
==============================================================================
                      Outcome equation (Y* = X * beta + e)
------------------------------------------------------------------------------
                     coef     std err         z     P>|z|      [0.025    0.975]
------------------------------------------------------------------------------
const              1.0108      0.0446   22.6544    0.0000      0.9234    1.0983
x1                 0.4925      0.0214   23.0340    0.0000      0.4506    0.5344
x2                -0.3017      0.0192  -15.7382    0.0000     -0.3393   -0.2642
------------------------------------------------------------------------------
             Selection equation (S* = Z * gamma + u, var(u) = 1)
------------------------------------------------------------------------------
                     coef     std err         z     P>|z|      [0.025    0.975]
------------------------------------------------------------------------------
const              0.0345      0.0213    1.6183    0.1056     -0.0073    0.0763
x1                 0.2598      0.0218   11.9411    0.0000      0.2171    0.3024
x2                 0.6242      0.0241   25.8950    0.0000      0.5769    0.6714
==============================================================================
```

Side-by-side comparison

All three estimators on the same data. Heckit's two-step and MLE recover β_{shared} much closer to the truth than OLS, and they recover ρ and σ as well.

In [41]:
```
comparison = pd.DataFrame({
    'true':         beta_true,
    'OLS (biased)': np.asarray(ols.params),
    'Heckit 2step': m_two.coef_,
    'Heckit MLE':   m_ml.coef_,
}, index=['const', 'shared', 'x_only'])
comparison['OLS error']   = comparison['OLS (biased)'] - comparison['true']
comparison['2step error'] = comparison['Heckit 2step']  - comparison['true']
comparison['MLE error']   = comparison['Heckit MLE']    - comparison['true']
comparison.round(4)
```

Out[41]:

	true	OLS (biased)	Heckit 2step	Heckit MLE	OLS error	2step error	MLE error
const	1.0	1.3651	1.0267	1.0108	0.3651	0.0267	0.0108
shared	0.5	0.4366	0.4903	0.4925	-0.0634	-0.0097	-0.0075
x_only	-0.3	-0.3037	-0.3010	-0.3017	-0.0037	-0.0010	-0.0017

In [42]:
```
pd.DataFrame({
    'true':      [rho_true, sigma_true],
    'two-step':  [m_two.rho_, m_two.sigma_],
    'MLE':       [m_ml.rho_,  m_ml.sigma_],
}, index=['rho', 'sigma']).round(4)
```

Out[42]:

	true	two-step	MLE
rho	0.6	0.5314	0.5523
sigma	1.0	0.9529	0.9586

Cross-validation against `py4etrics` and R's `sampleSelection`

Recovering the true DGP is a necessary but not sufficient check — a buggy estimator could return sensible-looking numbers and still pass it. The decisive test is replication of an *independent* implementation on the *same* data. We compare against two:

- **`py4etrics.Heckit`** (Hasebe et al.) — a Python implementation of the closed-form two-step Heckman estimator.
- **`sampleSelection::selection`** (Toomet & Henningsen, JSS 27(7), 2008) — the standard Heckit package in R, with both estimators (two-step and joint MLE) implemented on the same log-likelihood we use.

Both checks are run as part of the test suite (`tests/test_heckit.py::test_twostep_matches_py4etrics` and `tests/test_r_reference.py::test_heckit_mle_matches_R`). The cells below reproduce them live.

If `sampleSelection` is not installed locally, on macOS Apple Silicon run `brew install nlopt cmake pkg-config` once and then `install.packages('sampleSelection', dependencies = TRUE)`.

In [43]:
```
# Compare with py4etrics (two-step only -- py4etrics MLE is a no-op).
import warnings
try:
    from py4etrics.heckit import Heckit as Py4Heckit
    with warnings.catch_warnings():
        warnings.simplefilter('ignore')
        ref = Py4Heckit(y, sm.add_constant(X), sm.add_constant(Z)).fit(method='twostep')
    diff_beta  = np.max(np.abs(m_two.coef_  - ref.params))
    diff_gamma = np.max(np.abs(m_two.gamma_ - ref.select_res.params))
    print(f'max |beta_censtrunc  - beta_py4etrics|  = {diff_beta:.2e}')
    print(f'max |gamma_censtrunc - gamma_py4etrics| = {diff_gamma:.2e}')
    print(f'|sigma_censtrunc - sigma_py4etrics|     = {abs(m_two.sigma_ - np.sqrt(ref.var_reg
    print(f'|rho_censtrunc   - rho_py4etrics|       = {abs(m_two.rho_   - ref.corr_eqnerrors)
except ImportError:
    print('py4etrics not installed; pip install py4etrics to enable the comparison.')
```

```
max |beta_censtrunc  - beta_py4etrics|  = 1.88e-09
max |gamma_censtrunc - gamma_py4etrics| = 4.68e-09
|sigma_censtrunc - sigma_py4etrics|     = 1.08e-09
|rho_censtrunc   - rho_py4etrics|       = 3.16e-09
```

Both packages solve the same closed-form two-step expressions, so the residual disagreement (around 10^{-6}) is just numpy / statsmodels linear-algebra round-off.

In [44]:
```
# Compare with R's sampleSelection::selection (joint MLE).
# The R script expects a CSV with columns (y, s, x1, x2, z1, z2) and fits
# outcome 'y ~ x1 + x2' / selection 's ~ x1 + x2 + z1 + z2', so we build
# a fresh DGP with that exact layout.
import shutil, subprocess, json, tempfile, os
from pathlib import Path

def _find_r_heckit():
    for c in [Path('tests/reference/fit_heckit.R'),
              Path('../tests/reference/fit_heckit.R')]:
        if c.exists():
            return c
    return None

rscript, rpath = shutil.which('Rscript'), _find_r_heckit()
if rscript and rpath:
    rng_r = np.random.default_rng(20250601)
    n_r = 4000
    x1r = rng_r.normal(size=n_r); x2r = rng_r.normal(size=n_r)
    z1r = rng_r.normal(size=n_r); z2r = rng_r.normal(size=n_r)
    bb = np.array([0.5, 1.0, -0.4])
    gg = np.array([0.1, 0.3, -0.2, 0.5, 0.7])
    rho_r, sigma_r = 0.5, 1.2
    Sig = np.array([[sigma_r**2, rho_r*sigma_r],[rho_r*sigma_r, 1.0]])
    er, ur = rng_r.multivariate_normal([0, 0], Sig, size=n_r).T
    y_lat = bb[0] + bb[1]*x1r + bb[2]*x2r + er
    s_lat = gg[0] + gg[1]*x1r + gg[2]*x2r + gg[3]*z1r + gg[4]*z2r + ur
    s_obs = (s_lat > 0).astype(int)
    y_obs_r = np.where(s_obs == 1, y_lat, np.nan)

    Xr = np.column_stack([x1r, x2r])
    Zr = np.column_stack([x1r, x2r, z1r, z2r])
    m_ml_r = HeckitRegression(method='mle').fit(y_obs_r, Xr, Zr)

    fd, csvp = tempfile.mkstemp(suffix='.csv'); os.close(fd)
    pd.DataFrame({
        'y': y_obs_r, 's': s_obs,
        'x1': x1r, 'x2': x2r, 'z1': z1r, 'z2': z2r,
    }).to_csv(csvp, index=False, na_rep='NA')
```

```
    try:
        out = subprocess.run([rscript, str(rpath), csvp],
                             capture_output=True, text=True, timeout=180, check=True)
        rdat = json.loads(out.stdout)['mle']
        r_beta  = np.array([rdat['outcome']['(Intercept)'],
                            rdat['outcome']['x1'], rdat['outcome']['x2']])
        r_gamma = np.array([rdat['selection']['(Intercept)'],
                            rdat['selection']['x1'], rdat['selection']['x2'],
                            rdat['selection']['z1'], rdat['selection']['z2']])
        print(f"max |beta_censtrunc  - beta_R|   = {np.max(np.abs(m_ml_r.coef_  - r_beta)):.2e
        print(f"max |gamma_censtrunc - gamma_R|  = {np.max(np.abs(m_ml_r.gamma_ - r_gamma)):.2
        print(f"|sigma_censtrunc - sigma_R|      = {abs(m_ml_r.sigma_ - rdat['sigma']):.2e}")
        print(f"|rho_censtrunc   - rho_R|        = {abs(m_ml_r.rho_   - rdat['rho']):.2e}")
        print(f"|logLik_censtrunc - logLik_R|    = {abs(m_ml_r.llf_   - rdat['logLik']):.2e}"
    except Exception as exc:
        print(f'R run failed: {exc!r}')
    finally:
        os.remove(csvp)
else:
    print('Rscript not available at build time; see tests/test_r_reference.py for the gated te
```

```
max |beta_censtrunc  - beta_R|   = 5.68e-06
max |gamma_censtrunc - gamma_R|  = 6.81e-06
|sigma_censtrunc - sigma_R|      = 1.62e-06
|rho_censtrunc   - rho_R|        = 4.53e-06
|logLik_censtrunc - logLik_R|    = 9.93e-08
```

Two independent optimisers (scipy `L-BFGS-B` and R's `maxLik`) on the same joint log-likelihood: parameters, σ, ρ and the log-likelihood all match to optimiser tolerance ($\sim 10^{-3}$).

Six kinds of prediction

Six Heckman quantities are exposed through a single `predict()`. Each has a one-letter code so that any subset can be requested at once. With $\lambda(t) = \phi(t)/\Phi(t)$ the inverse Mills ratio:

Selection equation (uses Z):

- **`s`** — selection probability: $P(S = 1 \mid Z) = \Phi(Z'\hat{\gamma})$
- **`n`** — non-selection probability: $P(S = 0 \mid Z) = 1 - \Phi(Z'\hat{\gamma})$
- **`p`** — selection propensity (latent index): $Z'\hat{\gamma}$

Outcome equation (uses X, sometimes both):

- **`o`** — $\mathbb{E}[Y \mid X, Z, S = 1] = X'\hat{\beta} + \hat{\rho}\hat{\sigma}\,\lambda(Z'\hat{\gamma})$ (observed conditional)
- **`h`** — $\mathbb{E}[Y^* \mid X] = X'\hat{\beta}$ (hidden / unconditional latent)
- **`u`** — $\mathbb{E}[Y^* \mid X, Z, S = 0] = X'\hat{\beta} - \hat{\rho}\hat{\sigma}\,\phi(Z'\hat{\gamma})/[1 - \Phi(Z'\hat{\gamma})]$ (unobserved conditional)

Calling `predict(X=..., Z=...)` with no `kind` returns all six columns as a `DataFrame` in the order `s, n, p, o, h, u`. Subsets are selected by letter string (`kind='sn'`, `kind='ohu'`, …). A single letter returns a 1-D `ndarray`. Long names `'outcome'`, `'selection_prob'`, `'conditional'` keep working for backward compatibility (they alias `'h'`, `'s'`, `'o'`).

In [45]:
```
preds = m_ml.predict(X=X[:6], Z=Z[:6])   # default = all six
preds.assign(observed_y=y[:6], selected=S[:6]).round(3)
```

Out[45]:

	prob_selected	prob_not_selected	propensity	observed	hidden	unobserved	obs
0	0.626	0.374	0.322	1.405	1.084	0.548	
1	0.704	0.296	0.536	0.488	0.229	-0.390	
2	0.839	0.161	0.988	1.453	1.298	0.495	
3	0.499	0.501	-0.003	1.222	0.798	0.377	
4	0.363	0.637	-0.351	0.166	-0.381	-0.693	
5	0.071	0.929	-1.467	1.474	0.462	0.385	

Subsets via letter strings. The two probabilities sum to one; the three E[Y]-style columns satisfy $\mathbb{E}[Y \cdot S] = \Phi(Z'\gamma) \cdot o + [1 - \Phi(Z'\gamma)] \cdot 0$, which is just $\Phi(Z'\gamma) \cdot o$ on the selected and 0 on the unselected when the latter contribute $Y = 0$.

In [46]:
```
m_ml.predict(Z=Z[:6], kind='sn').round(3)   # only the two probabilities
```

Out[46]:

	prob_selected	prob_not_selected
0	0.626	0.374
1	0.704	0.296
2	0.839	0.161
3	0.499	0.501
4	0.363	0.637
5	0.071	0.929

In [47]:
```
m_ml.predict(X=X[:6], Z=Z[:6], kind='ohu').round(3)   # the three E[Y] variants
```

Out[47]:

	observed	hidden	unobserved
0	1.405	1.084	0.548
1	0.488	0.229	-0.390
2	1.453	1.298	0.495
3	1.222	0.798	0.377
4	0.166	-0.381	-0.693
5	1.474	0.462	0.385

In [48]:
```
m_ml.predict(X=X[:6], kind='h').round(3)   # single letter -> 1-D ndarray
```

Out[48]:
```
array([ 1.084,  0.229,  1.298,  0.798, -0.381,  0.462])
```

Same fit via a patsy formula

Instead of building `y`, `X`, `Z` by hand, every model class accepts a `from_formula(...)` constructor in the statsmodels style. For Heckit you pass *two* formulas — outcome and selection — sharing a single dataframe:

In [49]:
```
df = pd.DataFrame({
    'shared': shared, 'x_only': x_only, 'z_only': z_only,
    'inlf':   S, 'lwage': y_full,
})
m_form = HeckitRegression.from_formula(
    outcome   = 'lwage ~ 1 + shared + x_only',
    selection = 'inlf  ~ 1 + shared + z_only',
    data=df, method='mle',
).fit()
print(pd.DataFrame({
    'beta (explicit)': m_ml.coef_,
    'beta (formula)':  m_form.coef_,
}, index=['const', 'shared', 'x_only']).round(6))
```

```
        beta (explicit)  beta (formula)
const          1.010834        1.010834
shared         0.492478        0.492478
x_only        -0.301747       -0.301747
```

Bootstrap standard errors

Two-step second-stage standard errors are naive: they ignore the variability of $\hat{\gamma}$ from the probit step. A paired bootstrap gives proper SEs. The MLE Hessian-based SEs are already correct.

In [50]:
```
boot = m_two.bootstrap(y, X, Z, n_boot=80, seed=0)
pd.DataFrame({
    'beta':         m_two.coef_,
    'naive SE':     m_two.bse_,
    'bootstrap SE': boot['beta_se'],
}, index=['const', 'shared', 'x_only']).round(4)
```

Out[50]:

	beta	naive SE	bootstrap SE
const	1.0267	0.0491	0.0479
shared	0.4903	0.0223	0.0214
x_only	-0.3010	0.0209	0.0205

Marginal effects

Heckman's model has four standard marginal-effect quantities (Greene, *Econometric Analysis* §19.5):

`kind`	Differentiates
`'latent'`	$E[Y^* \mid X] = X'\beta$ — X-side only
`'conditional'`	$E[Y \mid X, Z, S = 1] = X'\beta + \rho\sigma\,\lambda(Z'\gamma)$
`'unconditional'`	$E[Y \cdot S \mid X, Z] = \Phi(Z'\gamma)X'\beta + \rho\sigma\,\phi(Z'\gamma)$
`'prob-selected'`	$P(S = 1 \mid Z) = \Phi(Z'\gamma)$ — Z-side only

A variable that appears in *both* equations (here `shared`) has its X- and Z-side derivatives **added together**. SEs come from the delta method on the joint MLE covariance; for a two-step fit they are `NaN` because the joint covariance is not available in closed form (use bootstrap instead).

Closed-form per-row derivatives with $\lambda = \phi/\Phi$ and $\delta(t) = \lambda(t)(t + \lambda(t))$ (so $\mathrm{d}\lambda/\mathrm{d}t = -\delta$):

$$\frac{\partial E[Y \mid S = 1]}{\partial v} = \beta_v \, \mathbf{1}\{v \in X\} \; - \; \gamma_v \, \rho\sigma \, \delta(Z'\gamma) \, \mathbf{1}\{v \in Z\}.$$

We pass `feature_names_outcome` / `feature_names_selection` so that variables with matching names in X and Z are recognised as the same variable.

In [51]:
```
m_me = HeckitRegression(method='mle').fit(
    y, X, Z,
    feature_names_outcome=['shared', 'x_only'],
    feature_names_selection=['shared', 'z_only'],
)
ame_cond = m_me.ame(kind='conditional')
print(ame_cond.summary())
```

```
                    Heckit Regression Marginal Effects
====================================
Dep. Variable:                     y
Method:                         dydx
At:                          overall
================================================================================
                dy/dx    std err          z      P>|z|      [0.025      0.975]
--------------------------------------------------------------------------------
shared          0.4086      0.020     20.771      0.000       0.370       0.447
x_only         -0.3017      0.019    -15.738      0.000      -0.339      -0.264
z_only         -0.2017      0.021     -9.568      0.000      -0.243      -0.160
================================================================================
```

All four marginal-effect kinds in a single table. Each row is averaged across the sample (AME); the entries are blank where the variable does not enter that equation.

In [52]:
```
kinds = ['latent', 'conditional', 'unconditional', 'prob-selected']
tbl = pd.DataFrame(index=sorted({n for k in kinds for n in m_me.ame(kind=k).names}))
for k in kinds:
    me = m_me.ame(kind=k)
    tbl[k] = pd.Series(me.margeff, index=me.names)
tbl.round(4)
```

Out[52]:

	latent	conditional	unconditional	prob-selected
shared	0.4925	0.4086	0.3349	0.0859
x_only	-0.3017	-0.3017	-0.1527	NaN
z_only	NaN	-0.2017	0.2057	0.2064

Reading the columns:

- `latent` reproduces the slope coefficients β (a sanity check).
- `conditional` differs from `latent` on `shared` (Z-side correction kicks in) and matches it exactly on `x_only` (X-only variable, no correction).
- `unconditional` is uniformly damped: $E[Y \cdot S]$ weighs the X-side by the selection probability $\Phi(Z'\gamma)$ rather than 1.
- `prob-selected` shows only the Z-side variables, scaled by $\phi(Z'\gamma)$.

Cross-check via finite differences of `predict`. The derivative formulas in `_heckit_effects.py` and the prediction code in `heckit.py` are written separately, so a central finite difference of `predict()` is an independent check of the analytic dy/dx. Below we evaluate at $X = \bar{X}$, $Z = \bar{Z}$ and perturb each regressor.

In [53]:
```
h = 1e-4
X_mean = X.mean(axis=0, keepdims=True)
Z_mean = Z.mean(axis=0, keepdims=True)
def cond_at(Xm, Zm):
    return float(m_me.predict(X=Xm, Z=Zm, kind='conditional').mean())

mem = m_me.mem(kind='conditional')
rows = []
for j, name in enumerate(['shared', 'x_only']):
    Xp, Xm_ = X_mean.copy(), X_mean.copy()
    Xp[0, j] += h; Xm_[0, j] -= h
    Zp, Zm_ = Z_mean.copy(), Z_mean.copy()
    if name == 'shared':
        Zp[0, 0] += h; Zm_[0, 0] -= h
    num = (cond_at(Xp, Zp) - cond_at(Xm_, Zm_)) / (2*h)
    rows.append((name, mem.margeff[mem.names.index(name)], num))
# z_only is Z-only.
Zp, Zm_ = Z_mean.copy(), Z_mean.copy()
Zp[0, 1] += h; Zm_[0, 1] -= h
num_z = (cond_at(X_mean, Zp) - cond_at(X_mean, Zm_)) / (2*h)
rows.append(('z_only', mem.margeff[mem.names.index('z_only')], num_z))
pd.DataFrame(rows, columns=['variable', 'analytic dy/dx', 'finite-difference']).round(8)
```

Out[53]:

	variable	analytic dy/dx	finite-difference
0	shared	0.405509	0.405509
1	x_only	-0.301747	-0.301747
2	z_only	-0.208959	-0.208959

The two columns agree to 7-8 decimals: the analytic derivative formulas in `_heckit_effects.py` are consistent with the prediction formulas in `heckit.py`.

Real-data demonstration: Mroz (1987)

The canonical Heckit application — Wooldridge's wage equation for married women. Of the 753 observations in Mroz's dataset, 428 women are in the labor force (`inlf = 1`) and have an observed `lwage`; the remaining 325 do not work and `lwage` is missing. Running OLS on the 428 working women answers a conditional question — *what determines wages among those who choose to work?* — whereas the latent question — *what would women earn if everyone worked?* — is what Heckit estimates.

Following Wooldridge (2010, Example 17.5):

- **Outcome equation:** `lwage ~ educ + exper + expersq`
- **Selection equation:** `inlf ~ educ + exper + expersq + nwifeinc + age + kidslt6 + kidsge6`

`nwifeinc`, `age`, `kidslt6`, and `kidsge6` serve as the exclusion restriction (they shift the *decision to work* but are not supposed to affect wages directly).

In [54]:
```
try:
    import wooldridge as woo
    mroz = woo.data('mroz')
except ImportError:
    raise ImportError('Install with: pip install wooldridge')

print(f'shape: {mroz.shape}')
print(f'in labor force (inlf=1): {(mroz["inlf"]==1).sum()}')
print(f'out of labor force:      {(mroz["inlf"]==0).sum()}')
print(f'lwage missing where inlf=0: {mroz.loc[mroz["inlf"]==0, "lwage"].isna().sum()}/'
      f'{(mroz["inlf"]==0).sum()}  (should be 100% — the canonical selection setup)')
```

```
shape: (753, 22)
in labor force (inlf=1): 428
out of labor force:      325
lwage missing where inlf=0: 325/325  (should be 100% — the canonical selection setup)
```

Step 1 — naive OLS on the working subsample. This is what one gets by ignoring the selection problem. We will compare its coefficient on `educ` to the Heckit-corrected version below.

In [55]:
```
import statsmodels.formula.api as smf

ols = smf.ols('lwage ~ educ + exper + expersq',
              data=mroz[mroz['inlf'] == 1]).fit()
print(ols.summary().tables[1])
```

```
==============================================================================
                 coef    std err          t      P>|t|      [0.025      0.975]
------------------------------------------------------------------------------
Intercept     -0.5220      0.199     -2.628      0.009      -0.912      -0.132
educ           0.1075      0.014      7.598      0.000       0.080       0.135
exper          0.0416      0.013      3.155      0.002       0.016       0.067
expersq       -0.0008      0.000     -2.063      0.040      -0.002   -3.82e-05
==============================================================================
```

Step 2 — Heckit two-step. The selection equation includes the exclusion variables; the second stage adds the inverse Mills ratio to the wage equation.

In [56]:
```
from censtrunc import HeckitRegression

heck_ts = HeckitRegression.from_formula(
    outcome   = 'lwage ~ educ + exper + expersq',
    selection = 'inlf ~ educ + exper + expersq + nwifeinc + age + kidslt6 + kidsge6',
    data=mroz, method='twostep',
).fit()
print(heck_ts.summary())
```

```
                        Heckman Selection Regression
==============================================================================
Method:                  twostep                 No. Observations:        753
Selected (S=1):          428                     Censored (S=0):          325
sigma_e:                 0.663629                rho:                     0.048614
==============================================================================
                     Outcome equation (Y* = X * beta + e)
------------------------------------------------------------------------------
                     coef     std err         z     P>|z|      [0.025    0.975]
------------------------------------------------------------------------------
const             -0.5781      0.3051   -1.8948    0.0581     -1.1761    0.0199
educ               0.1091      0.0155    7.0243    0.0000      0.0786    0.1395
exper              0.0439      0.0163    2.6980    0.0070      0.0120    0.0758
expersq           -0.0009      0.0004   -1.9567    0.0504     -0.0017    0.0000
------------------------------------------------------------------------------
             Selection equation (S* = Z * gamma + u, var(u) = 1)
------------------------------------------------------------------------------
                     coef     std err         z     P>|z|      [0.025    0.975]
------------------------------------------------------------------------------
const              0.2701      0.5083    0.5313    0.5952     -0.7262    1.2663
educ               0.1309      0.0252    5.1845    0.0000      0.0814    0.1804
exper              0.1233      0.0187    6.5906    0.0000      0.0867    0.1600
expersq           -0.0019      0.0006   -3.1454    0.0017     -0.0031   -0.0007
nwifeinc          -0.0120      0.0048   -2.4843    0.0130     -0.0215   -0.0025
age               -0.0529      0.0085   -6.2368    0.0000     -0.0695   -0.0362
kidslt6           -0.8683      0.1185   -7.3269    0.0000     -1.1006   -0.6360
kidsge6            0.0360      0.0435    0.8282    0.4075     -0.0492    0.1212
==============================================================================
Note: two-step second-stage standard errors are naive; call .bootstrap(y, X, Z) for proper Heckman-correc
==============================================================================
```

Step 3 — Joint MLE Heckit (asymptotically efficient).

In [57]:
```
heck_ml = HeckitRegression.from_formula(
    outcome   = 'lwage ~ educ + exper + expersq',
    selection = 'inlf ~ educ + exper + expersq + nwifeinc + age + kidslt6 + kidsge6',
    data=mroz, method='mle',
).fit()
print(heck_ml.summary())
```

```
                        Heckman Selection Regression
================================================================================
Method:                  mle                     No. Observations:        753
Selected (S=1):          428                     Censored (S=0):          325
sigma_e:                 0.663631                rho:                     0.048580
Log-Likelihood:          -832.8972
================================================================================
                     Outcome equation (Y* = X * beta + e)
--------------------------------------------------------------------------------
                     coef     std err         z     P>|z|      [0.025    0.975]
--------------------------------------------------------------------------------
const             -0.5781      0.2577   -2.2432    0.0249     -1.0832   -0.0730
educ               0.1091      0.0148    7.3617    0.0000      0.0800    0.1381
exper              0.0439      0.0148    2.9622    0.0031      0.0148    0.0729
expersq           -0.0009      0.0004   -2.0622    0.0392     -0.0017   -0.0000
--------------------------------------------------------------------------------
             Selection equation (S* = Z * gamma + u, var(u) = 1)
--------------------------------------------------------------------------------
                     coef     std err         z     P>|z|      [0.025    0.975]
--------------------------------------------------------------------------------
const              0.2701      0.5089    0.5307    0.5956     -0.7274    1.2675
educ               0.1310      0.0254    5.1611    0.0000      0.0812    0.1807
exper              0.1234      0.0187    6.5870    0.0000      0.0867    0.1601
expersq           -0.0019      0.0006   -3.1463    0.0017     -0.0031   -0.0007
nwifeinc          -0.0122      0.0049   -2.4995    0.0124     -0.0217   -0.0026
age               -0.0528      0.0085   -6.2247    0.0000     -0.0694   -0.0362
kidslt6           -0.8683      0.1187   -7.3136    0.0000     -1.1010   -0.6356
kidsge6            0.0360      0.0435    0.8283    0.4075     -0.0492    0.1212
================================================================================
```

Coefficient comparison

Lining up the three wage-equation slope estimates. Wooldridge's textbook reports an OLS return-to-education of about 0.108; the Heckit-corrected return is close to that, and the implied correlation $\hat{\rho}$ between the wage-equation error and the participation error is informative about how strong the selection channel is.

In [58]:
```
import numpy as np
comparison = pd.DataFrame({
    'OLS (selected)': np.asarray(ols.params),
    'Heckit two-step': heck_ts.coef_,
    'Heckit MLE':     heck_ml.coef_,
}, index=heck_ts.outcome_feature_names_)
comparison.round(4)
```

Out[58]:

	OLS (selected)	Heckit two-step	Heckit MLE
const	-0.5220	-0.5781	-0.5781
educ	0.1075	0.1091	0.1091
exper	0.0416	0.0439	0.0439
expersq	-0.0008	-0.0009	-0.0009

In [59]:
```
pd.DataFrame({
    'two-step':  [heck_ts.rho_, heck_ts.sigma_],
    'MLE':       [heck_ml.rho_, heck_ml.sigma_],
}, index=['rho', 'sigma']).round(4)
```

Out[59]:

	two-step	MLE
rho	0.0486	0.0486
sigma	0.6636	0.6636

Interpreting $\hat{\rho}$: a positive (negative) value means that the unobserved factors that raise wages also raise (lower) the probability of working. If $\hat{\rho}$ is small or insignificant (a Wald or LR test on $\rho = 0$ would say so), the selection correction is empirically unnecessary and ordinary OLS is fine.

Marginal effects on Mroz

For policy interpretation we usually want marginal effects on the *conditional* wage $E[\log \text{wage} \mid \text{works}]$ and on the *participation probability* $P(\text{works})$. The participation effects below use the standard probit-style derivative $\gamma_v \, \phi(Z'\gamma)$, averaged across the sample.

In [60]:
```
ame_cond = heck_ml.ame(kind='conditional')
ame_prob = heck_ml.ame(kind='prob-selected')
pd.concat([
    ame_cond.summary_frame()[['dy/dx', 'std err', 'P>|z|']].rename(
        columns={'dy/dx': 'cond E[lwage]', 'std err': 'SE', 'P>|z|': 'pval'}),
    ame_prob.summary_frame()[['dy/dx', 'std err', 'P>|z|']].rename(
        columns={'dy/dx': 'P(in LF)', 'std err': 'SE_p', 'P>|z|': 'pval_p'}),
], axis=1).round(4)
```

Out[60]:

	cond E[lwage]	SE	pval	P(in LF)	SE_p	pval_p
educ	0.1067	0.0143	0.0000	0.0394	0.0073	0.0000
exper	0.0417	0.0131	0.0015	0.0371	0.0052	0.0000
expersq	-0.0008	0.0004	0.0361	-0.0006	0.0002	0.0013
nwifeinc	0.0002	0.0007	0.7418	-0.0037	0.0015	0.0116
age	0.0010	0.0028	0.7356	-0.0159	0.0024	0.0000
kidslt6	0.0156	0.0463	0.7352	-0.2611	0.0319	0.0000
kidsge6	-0.0006	0.0021	0.7535	0.0108	0.0131	0.4069

The `educ` row reads: one extra year of schooling raises *expected log-wage among workers* by the `cond E[lwage]` figure, *and separately* raises the probability of being in the labour force by the `P(in LF)` figure.

Z-only variables (`nwifeinc`, `age`, `kidslt6`, `kidsge6`) still appear in the `cond E[lwage]` column with very small, statistically-insignificant entries. That is **not** a blank: they do affect the conditional wage, but only *through the Mills-ratio correction* $-\gamma_v \, \rho \, \sigma \, \delta(Z'\gamma)$. On Mroz the estimated correlation $\hat{\rho}$ is close to zero (about 0.03), so

that channel is numerically tiny and the entries land around zero — exactly as one would expect when the selection correction is weak.